Ahmedabad University

CSE213: Digital Logic with Hardware Description Language

GROUP PROJECT

2048 GAME

Group - 16

Submitted to faculty: Harmeet Kaur

Date of Submission: 21st November 2025

Student Details

Roll No.	Name of the Student	Name of the Program
AU2440104	Rinal Shah	BTech CSE
AU2440010	Jahnavi Panchal	BTech CSE
AU2440005	Dhruti Kobawala	BTech CSE
AU2440032	Krishna Patel	BxMx

1. Introduction

1.1 Overview of the Project

This project implements the popular 2048 puzzle game using a hybrid design of Verilog for the game logic and Python (Tkinter) for the graphical user interface (GUI). The Verilog module performs tile movement, merging operations, random tile generation, score management, and win/lose checks, while the Python GUI displays the board and captures user inputs.

1.2 Motivation and Significance

The motivation behind the project is to demonstrate how hardware description languages like Verilog can be used to build complex logic-based games and interface them with software-based GUIs. This showcases hardware–software co-design techniques, supports educational learning, and illustrates interactive digital system design.

1.3 Objectives and Scope

Objectives:

- Implement a fully functional 2048 game engine in Verilog.
- Interface Verilog simulation output with a Python GUI.
- Provide real-time interaction using keyboard inputs.
- Ensure correctness of tile merging, compressing, and score updates.

Scope:

The project covers simulation-level implementation of the game without deploying onto an FPGA. Python is used only for visualization and input, while all game logic remains in Verilog.

2. System Design and Architecture

2.1 Overall System Description

The system consists of two main components: the Verilog-based 2048 game engine and a Python GUI frontend. The Verilog module handles all game rules, tile operations, random tile spawning, and win/lose conditions. The Python program communicates via stdin/stdout with the Verilog process running under Icarus Verilog (vvp).

2.2 Block Diagram

```
┌──────────────────────────────────────────────────────────┐
│                     PYTHON APPLICATION                   │
│  ┌──────────────────┐          ┌──────────────────┐      │
│  │ Graphical UI (GUI)│          │ Runtime Input Taker│     │
│  │ - Renders board  │◄────────►│ - Sends WASD     │       │
│  │ - Shows score    │          │ - Sends Quit (Q) │       │
│  └──────────────────┘          └──────────────────┘       │
```

(ASCII: w/a/s/d/q)

VERILOG GAME CORE

CONTROL / FSM LOGIC

- Initial reset
- Input loop + validation
- Win/Lose detection

MOVE SYSTEM

(2048 game mechanics implementation)

- move_left() - move_right()
- move_up() - move_down()
- compress_left(), merge_left()
- mirror_h(), transpose()

RANDOM TILE ENGINE

spawn_random_tile()

- Scans empty blocks
- Random index selection
- Inserts tile (2 or 4)

BOARD MEMORY ARRAY

board[0:3][0:3] : current grid state

score : integer

OUTPUT INTERFACE

print_board_machine()

```
│  │  $display SCORE / WIN / LOSE          │  │
│  │  $fflush → sent to Python             │  │
│  └──────────────────────────────────────┘  │
└──────────────────────────────────────────┘
```

2.3 Component Description

• Verilog Game Engine: Handles board representation, tile merging logic, movement, score updates, random tile generation, and win/lose checking.
• Python GUI: Displays the game board using Tkinter labels, updates tile colors, and sends user commands to Verilog.
• Icarus Verilog: Used to compile and simulate the Verilog code.

3. Verilog Code Implementation

3.1 Module Structure

The Verilog code contains a manipulation of left, right, up and down movement of the tiles. The design applies procedural blocks, tasks, and functions to apply game logic to the system without the need to control it with FSMs.

3.2 Key Components

1. Module Overview

- **Module name:** game2048
- **Language:** Verilog (simulation), interfaced with Python GUI
- **Purpose:** Implement 2048 game logic, communicate with Python GUI for input/output.

Core components:

- board[4][4] → stores tile values (integers)
- score → stores player score
- seed → RNG seed
- Input/output via stdin/stdout ($fgetc, $display)

Bit structure:

- Each tile stored as **integer** (0 = empty, powers of 2 for tiles)
- ch as reg [7:0] to store ASCII input character

2. Initial Block

Code snippet:

```
None
initial begin
```

```
    seed = 32'h12345678;
    score = 0;
    #100_000; // wait for Python GUI
    for (i=0;i<4;i=i+1)
        for (j=0;j<4;j=j+1)
            board[i][j] = 0;
    spawn_random_tile();
    spawn_random_tile();
    print_board_machine();
    $display("SCORE %0d", score);
    $fflush();
end
```

Explanation:

- Initializes RNG seed, score, and clears the 4×4 board.
- Spawns **two initial random tiles** (2 or 4).
- Prints board and score in **machine-readable format**.
- **State:** board empty → board with 2 tiles.
- **Bit usage:** 32-bit seed, integer for board tiles, reg [7:0] ch for input.

3. Input Loop

Code snippet:

```
None
r = $fgetc(32'h8000_0000); // read character from Python GUI
case (ch)
    "w": move_up();
    "a": move_left();
    "s": move_down();
    "d": move_right();
    "q": $finish;
endcase
```

Explanation:

- Waits for key input from Python (w, a, s, d for moves, q to quit).
- Performs the corresponding **move**.
- **State transition:** after valid move → board updated → spawn new tile → check win/lose.
- **Bit usage:** ch is 8-bit ASCII code of input.

---

4. Random Tile Spawning

Code snippet:

```
None
tile = 2;
if (($random(seed) & 3) == 0) tile = 4;
board[i][j] = tile;
```

Explanation:

- Selects an **empty cell** from a list of empty positions.
- Tile is either **2 (75%)** or **4 (25%)** using $random & bitmask.
- **State transition:** empty cell → occupied by 2 or 4.
- **Bit usage:** RNG produces 32-bit integers; last 2 bits used for probability.

5. Move Logic

Code snippet (move_left example):

```
None
task move_left;
   begin
      compress_left(); // slide tiles
      merge_left();    // merge equal tiles
      compress_left(); // slide again after merge
   end
endtask
```

Explanation:

- **compress_left:** slide tiles to remove gaps
- **merge_left:** merge adjacent equal tiles, update score
- **move_right/up/down:** reuse move_left via mirror_h() and transpose()
- **State transitions:** before move → after compress → after merge → after compress
- **Bit usage:** integer for tiles, sum stored in score (integer)

6. Board Transformation Tasks

Code snippet:

```
None
task mirror_h;    // flip horizontally
task transpose;   // swap rows and columns
```

Explanation:

- mirror_h: left ↔ right for move_right

- transpose: swap rows/columns for move_up/down
- Enables **reusing move_left logic** for all directions.

7. State Transitions Overview

Event	State Before	State After
Initial Block	Empty board	2 random tiles placed
Valid Move	Board with tiles	Tiles shifted/merged, new tile spawned
Merge	Two equal tiles adjacent	Merged tile doubles, score updated
Game Over	Board full, no merges	LOSE
Win	Tile = 2048	WIN

8. Structure Summary

- **Top-level module:** game2048
- **Tasks:** spawn_random_tile, compress_left, merge_left, move_left/right/up/down, mirror_h, transpose
- **Functions:** check_win, check_game_over
- **Variables:** board[4][4], score, seed, ch
- **I/O:** stdin ($fgetc), stdout ($display)
- **Interaction:** Python GUI reads machine-readable output and sends input keys

3.3 Design Decisions

Board storage has been implemented by a plain integer array. Tasks and repetitive operations diminish duplication of codes. Other directional movements (right, up, down) are achieved by reapplying logic of left-moving as well as changing the board such as transposing or mirroring it.

4. Simulation and Testing

4.1 Testbench Approach

It has a Python-based interactive testbench that substitutes a conventional Verilog testbench. vvp is used to run the simulator and Python can input and read output of Verilog. This enables real time simulation and proving of the logic.

4.2 Simulation Results

The game was played by means of different order of moves. Board updates and score increments had been checked visually on the GUI. Win and lose conditions were also established.

4.3 Waveform Analysis

Waveform analysis is not required because this project is interactive. Internal signals e.g. the values of board tiles can be recorded with a VCD file when necessary.

5. Results and Discussion

The project manages to show how to apply a full interactive game engine in Verilog. The Python GUI links with the Verilog simulation without any problem. Movement and merging logic act as desired and the random tile generation takes place as desired after each move.

5.1 Performance Analysis

- Timing: In simulation, everything happens immediately since it is more of a procedure.
- Area: The design is small and would consume few resources of FPGA in case of a synthesis.

5.2 Improvements & Future Work

- Play automatically with an AI solver.
- Use PyGame to implement smooth animations as opposed to the Tkinter.
- Move the Verilog code over to FPGA to play the game on hardware.
- Add high score tracking.

5.3 Real-world Relevance

This project illustrates the integration of hardware logic and software interfaces, which is a major concept in embedded system design, digital logic instruction and co-design applications of hardware-software interfaces.